How To Prepare

Computer Science Paper

For PGEE 2026?

Complete Subject-wise Preparation Guide · Based on PGEE 2016 – 2025 Analysis · IIIT Hyderabad

01 **Digital Logic**
~6 Qs · Every Year

02 **C Programming**
~7 Qs · Every Year

03 **Data Structures**
~6 Qs · Every Year

04 **Algorithms & Problem Solving**
~5 Qs · Every Year

05 **Discrete Math & Probability**
~5 Qs · Every Year

06 **Relational Databases**
~5 Qs · 6 of 8 yrs

07 **Computer Networks**
~5 Qs · Every Year

08 **Operating Systems**
~6 Qs · Every Year

No Negative Marking · Attempt Every Question · GATE PYQs Appear Verbatim

Exam Overview & Preparation Strategy

PGEE 2026 Expected Exam Pattern

Section	Questions	Marks	Difficulty	Duration
General Aptitude	40	40	Easy to Moderate	~50 min
Computer Science	40	40	Moderate to Hard	~70 min
Engineering Maths	10	10	Moderate	~20 min
TOTAL	90	90	—	~140 min

Note: No negative marking in PGEE. Attempt every single question — a blank guarantees zero, while even a random guess gives positive expected value.

Subject-wise Expected Weightage (CS Section)

Subject	Expected Qs	Priority	Appeared (last 8 yrs)
C Programming	6 – 8	MUST	All 8 years
Data Structures	5 – 7	MUST	All 8 years
Digital Logic	5 – 7	MUST	All 8 years
Operating Systems	4 – 6	MUST	All 8 years
Computer Networks	4 – 5	MUST	All 8 years
Algorithms & Problem Solving	4 – 5	MUST	All 8 years
Discrete Math & Probability	4 – 5	HIGH	7 of 8 years
Relational Databases	4 – 5	HIGH	6 of 8 years

8-Week Preparation Strategy

Week 1 – 2

- Revise C Programming — output tracing, pointers, storage classes, memory functions.
- Cover Data Structures — arrays, linked lists, stacks, queues, trees, heaps, hashing.
- Solve 50+ output tracing questions daily from GeeksForGeeks.

Week 3 – 4

- Cover Digital Logic — Boolean algebra, flip-flops, counters, K-maps, MUX.
- Cover Algorithms — sorting, graph algorithms (Dijkstra, Prim, Kruskal), DP basics.
- Solve 30+ GATE PYQs from these two subjects.

Week 5 – 6	◆ Cover DBMS — normalization, SQL queries, functional dependencies, transactions. ◆ Cover Operating Systems — scheduling, memory management, page replacement, semaphores. ◆ Revise Discrete Math — logic, counting, probability, graph theory.
Week 7 – 8	◆ Cover Computer Networks — subnetting, TCP/IP, protocols, throughput formulas. ◆ Take 2 full mock tests per week. Review every wrong answer deeply. ◆ Revise all formula sheets. GATE PYQ revision — look for repeated questions.

KEY STRATEGY TIPS

- GATE PYQs are a goldmine — 4 to 5 questions appear verbatim (even options unchanged) every year.
- Do NOT skip any subject — topic distribution is unpredictable year to year.
- IndiaBix.com for aptitude practice | GateOverflow.in for CS theory and PYQs.
- Scientific calculator is allowed — similar to GATE calculator.
- Last 2 weeks: only revision and mock tests. Stop learning new topics.

01 Digital Logic

Boolean Algebra · Flip-Flops · Counters · Combinational Circuits

Digital Logic appears in **every single year** of PGEE, with 5 to 8 questions. PGEE 2023 was especially Digital Logic heavy. Questions are highly predictable — they test specific formulas and definitions. Mastering this subject gives guaranteed marks.

Boolean Algebra — Core Laws

Law	AND Form	OR Form
Identity	A . 1 = A	A + 0 = A
Null	A . 0 = 0	A + 1 = 1
Complement	A . A' = 0	A + A' = 1
Idempotent	A . A = A	A + A = A
Involution	A'' = A	(same)
Commutative	A . B = B . A	A + B = B + A
Associative	(A.B).C = A.(B.C)	(A+B)+C = A+(B+C)
Distributive	A.(B+C) = A.B + A.C	A+B.C = (A+B).(A+C)
De Morgan	(A.B)' = A' + B'	(A+B)' = A' . B'
Absorption	A.(A+B) = A	A + A.B = A
Consensus	A.B + A'.C + B.C = A.B + A'.C	—

Flip-Flop Characteristic Equations

Type	Characteristic Equation	Special State	Use Case
SR	Q* = S + R'.Q (SR=0)	SR=1 is INVALID	Basic latch, simple storage
D	Q* = D	None	Registers, data capture
JK	Q* = J.Q' + K'.Q	J=K=1 toggles	Universal — most common in PGEE
T	Q* = T XOR Q	T=0: hold, T=1: toggle	Counters, frequency division

Counter Design Rules

Counter Rules

- Mod-N counter needs ceil(log2 N) flip-flops
- Mod-10 (decade counter) = 4 FFs (2^4 = 16 >= 10)
- Mod-50 counter = 6 FFs (2^6 = 64 >= 50)
- Mod-100 counter = 7 FFs (2^7 = 128 >= 100)
- Ripple counter: asynchronous, propagation delay
- Synchronous counter: all FFs clocked simultaneously

Counter Types & Examples

- Ring counter: N FFs produce N distinct states
- Johnson (twisted ring): 2N states for N FFs
- Up counter: Q toggles when all lower bits = 1
- Down counter: Q toggles when all lower bits = 0
- Counter frequency = Clock / Modulus
- Sequence detector: use D/JK FFs with next-state logic

Combinational Circuits Reference

Circuit	Inputs	Outputs	Key Equations / Notes
Half Adder	A, B	S, Cout	Sum = A XOR B | Carry = A AND B
Full Adder	A,B,Cin	S, Cout	S = A XOR B XOR Cin | Cout = AB + BCin + ACin
Half Subtractor	A, B	D, Bor	Diff = A XOR B | Borrow = A'.B
n-to-2^n Decoder	n	2^n	Requires 2^n AND gates (one per output line)
2^n-to-n Encoder	2^n	n	Requires n OR gates (one per output bit)
2:1 MUX	2 + 1sel	1	Y = I0.S' + I1.S — implements any 2-var function
4:1 MUX	4 + 2sel	1	2 select lines; can implement any 3-var Boolean function
K-map (n vars)	2^n cells	—	Group 1s in powers of 2: 1,2,4,8,16 — always wrap edges

Most Frequently Asked in PGEE

- JK flip-flop characteristic equation — Q* = J.Q' + K'.Q (appears almost every year)
- Mod-N counter: minimum flip-flops = ceiling of log base 2 of N
- Johnson counter has 2N states for N flip-flops (not just N)
- De Morgan's theorem: (A.B)' = A' + B' and (A+B)' = A'.B'
- Identity element of NOR gate is 0 (NOR(A, 0) = A' — not identity, it inverts!)
- Boolean simplification: X(X+Y) = X | X + X.Y = X | (A'.B')(A+B) = 0
- Full adder sum and carry equations — asked directly in PGEE 2023

02 C Programming

Output Tracing · Pointers · Memory · Storage Classes · Strings

C Programming is the **highest-weighted subject** in PGEE. PGEE 2022 alone had 8–9 questions. Questions focus on output tracing, pointer behavior, storage classes, and tricky expressions. The key skill is reading code carefully and predicting exactly what gets printed.

Storage Classes — The Most Important Table

Class	Where Stored	Default Value	Scope	Lifetime
auto	Stack	Garbage	Local block	Until block ends
register	CPU Register	Garbage	Local block	Until block ends
static	Data Segment (BSS)	Zero (0)	Local or file-level	Entire program run
extern	Data Segment	Zero (0)	Global (across files)	Entire program run

Key fact: static local variables retain their value between function calls because they live in the data segment, not the stack.

Operator Precedence (Highest to Lowest)

Level	Operators	Associativity
1 (Highest)	( ) [ ] -> . (postfix++ --)	Left to Right
2	! ~ ++ -- + - (unary) * (deref) & (addr) sizeof cast	Right to Left
3	* (mult) / %	Left to Right
4	+ (add) - (sub)	Left to Right
5	<< >>	Left to Right
6	< <= > >=	Left to Right
7	== !=	Left to Right
8	& (bitwise AND)	Left to Right
9	^ (bitwise XOR)	Left to Right
10	| (bitwise OR)	Left to Right
11	&&	Left to Right
12	||	Left to Right
13	?: (ternary)	Right to Left
14 (Lowest)	= += -= *= /= %= etc.	Right to Left

Classic PGEE Trap Questions — Know These Cold

Expression or Concept	Answer / Result	Explanation
printf("hello")	Returns int (5)	printf returns the number of characters written
calloc(n, size)	Returns void*	Allocates n*size bytes, initialised to zero
malloc(size)	Returns void*	Allocates size bytes, content is garbage
--10	No decrement happens	Parsed as -(-10) = 10; cannot decrement a literal
int x = --10	x = 10	Same reason above — no variable to decrement
static int x = 0	x persists across calls	Static local retains value between invocations
arr++ (array name)	Compile error	Array name is a constant pointer, cannot increment
****arr (2D array)**	arr[0][0]	Double dereference of pointer-to-pointer
***(arr + i)**	Same as arr[i]	Array indexing is exactly pointer arithmetic
sizeof('A') in C	4 bytes	Character literal is int in C (not char)
sizeof("hi")	3 bytes	String literal includes null terminator
fork() — child gets	Return value 0	Parent gets child PID; child gets 0; error = -1
n fork() calls	2^n - 1 child processes	Each fork doubles the number of processes

Memory Management Functions

Function	Full Signature	Init to Zero?	Notes
malloc(size)	void* malloc(size_t size)	No	Returns NULL on failure
calloc(n, size)	void* calloc(size_t n, size_t size)	YES	n elements of given size
realloc(ptr, size)	void* realloc(void* ptr, size_t size)	No	Preserves old content
free(ptr)	void free(void* ptr)	N/A	Must match malloc/calloc

Pointer Rules — Quick Reference

Pointer Basics

- int *p — p points to an integer
- int **p — pointer to a pointer to int
- *p — dereference (get value at address)
- &x; — address-of operator (get address of x)
- p++ — advances pointer by sizeof(int) bytes
- NULL pointer dereference = undefined behavior
- void* holds any pointer type without casting

Advanced Pointer Concepts

- const int *p — cannot modify *p (value)
- int * const p — cannot modify p (address)
- Function ptr: int (*f)(int, int) = &func;
- arr[i][j] = *(*(arr+i)+j) — 2D equivalent
- Pointer comparison: valid only within same array
- p - q = number of elements between p and q
- sizeof pointer = 4 (32-bit) or 8 (64-bit) bytes

03 Data Structures

Arrays · Trees · Heaps · Hashing · Graphs · Stacks & Queues

Data Structures is tested every year with questions on time complexity, tree operations, hashing techniques, and graph representations. You must be able to **trace algorithms on small examples** and know every complexity by heart.

Time Complexity — Complete Reference Table

Data Structure	Access	Search	Insert	Delete	Space
Array	O(1)	O(n)	O(n)	O(n)	O(n)
Singly Linked List	O(n)	O(n)	O(1)*	O(1)*	O(n)
Stack (Array/LL)	O(n)	O(n)	O(1)	O(1)	O(n)
Queue (Array/LL)	O(n)	O(n)	O(1)	O(1)	O(n)
Binary Search Tree	O(h)	O(h)	O(h)	O(h)	O(n)
AVL Tree	O(log n)	O(log n)	O(log n)	O(log n)	O(n)
Min/Max Heap	O(n)	O(n)	O(log n)	O(log n)	O(n)
Hash Table (avg)	O(1)	O(1)	O(1)	O(1)	O(n)
Hash Table (worst)	O(n)	O(n)	O(n)	O(n)	O(n)
Graph (Adj Matrix)	O(1)	O(V^2)	O(1)	O(1)	O(V^2)
Graph (Adj List)	O(degree)	O(V+E)	O(1)	O(E)	O(V+E)

** O(1) insert/delete only at known node position*

Trees — Essential Properties

Binary Tree Facts

- ◆ Full BT: every node has 0 or 2 children
- ◆ Complete BT: all levels full except last (filled left-to-right)
- ◆ Perfect BT: all internal nodes have 2 children, all leaves same level
- ◆ Height h BST — min nodes: h+1 | max nodes: 2^(h+1) - 1
- ◆ BST inorder traversal always produces a sorted sequence
- ◆ BST deletion has 3 cases: leaf, 1 child, 2 children
- ◆ For 2-children delete: replace with inorder successor or predecessor

AVL & Heap Properties

- ◆ AVL Tree: balance factor = |height(left) - height(right)| <= 1
- ◆ AVL minimum nodes: N(h) = N(h-1) + N(h-2) + 1 (Fibonacci-like)
- ◆ AVL rotations: LL needs right rotation | RR needs left rotation
- ◆ LR needs left then right | RL needs right then left
- ◆ Heap: parent <= children (min-heap) | parent >= children (max-heap)
- ◆ Build heap from array: O(n) — start from index n/2, sift down
- ◆ BSTs with n distinct keys = Catalan(n) = C(2n,n)/(n+1)

Hashing Techniques Comparison

Technique	Probe Formula	Primary Clustering?	Secondary?
Linear Probing	h(k,i) = (h(k) + i) mod m	YES	No
Quadratic Probing	h(k,i) = (h(k) + i^2) mod m	No	YES
Double Hashing	h(k,i) = (h1(k) + i * h2(k)) mod m	No	No
Separate Chaining	h(k) mod m — each slot is a list	N/A	N/A

Key rule: Double hashing eliminates both clustering types — it is the best open-addressing method for uniform distribution.

Graph Traversal & Representation

Operation	Adjacency Matrix	Adjacency List	Which is Better?
Space	O(V^2)	O(V + E)	List for sparse graphs
Check edge (u, v)	O(1)	O(degree(u))	Matrix for dense graphs
All neighbours of u	O(V)	O(degree(u))	List is much faster
BFS time complexity	O(V^2)	O(V + E)	List always preferred
DFS time complexity	O(V^2)	O(V + E)	List always preferred
Detect cycle (undirected)	O(V+E)	O(V + E)	Both work; use Union-Find
Topological sort	O(V^2)	O(V + E)	List + Kahn's or DFS

04 Algorithms & Problem Solving

Sorting · Searching · Graph Algorithms · Dynamic Programming · Recurrences

Algorithm questions in PGEE test both theoretical knowledge (time complexities, algorithm correctness) and practical tracing (running an algorithm on a small graph or array). Graph algorithms, sorting, and DP problem identification appear every year.

Sorting Algorithms — Complete Complexity Table

Algorithm	Best Case	Average	Worst Case	Space	Stable ?
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	No
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting Sort	O(n + k)	O(n + k)	O(n + k)	O(k)	Yes
Radix Sort	O(d(n+k))	O(d(n+k))	O(d(n+k))	O(n+k)	Yes

Merge Sort is the best sort for Linked Lists (O(n log n), O(log n) space). Quick Sort worst case O(n^2) happens with sorted input and bad pivot selection.

Graph Algorithms — Time Complexities & Use Cases

Algorithm	With Array	With Min-Heap / PQ	Handles Neg. Wts?	Use Case
Dijkstra	O(V^2)	O((V+E) log V)	NO	SSSP, non-negative weights
Bellman-Ford	O(V.E)	O(V.E)	YES	SSSP with negative weights
Floyd-Warshall	O(V^3)	O(V^3)	YES	All-pairs shortest path
Prim's MST	O(V^2)	O(E log V)	N/A	MST, better for dense graphs
Kruskal's MST	O(E log E)	O(E log E)	N/A	MST, better for sparse graphs
BFS	O(V+E)*	—	N/A	SSSP unweighted, level order
DFS	O(V+E)*	—	N/A	Cycle detect, topo sort, SCC
Max Flow (EK)	O(V.E^2)	O(V.E^2)	N/A	Edmonds-Karp uses BFS

** O(V+E) assumes Adjacency List; O(V^2) with Adjacency Matrix*

Master Theorem — T(n) = a T(n/b) + f(n)

Case	Condition	Solution	Classic Example
1	f(n) = O(n^c) where c < log_b(a)	T(n) = Theta(n^log_b(a))	T(n)=2T(n/2)+1 -> O(n)
2	f(n) = Theta(n^c) where c = log_b(a)	T(n) = Theta(n^c . log n)	T(n)=2T(n/2)+n -> O(n log n)
3	f(n) = Omega(n^c) where c > log_b(a)	T(n) = Theta(f(n))	T(n)=T(n/2)+n -> O(n)

Dynamic Programming — Classic Problems

Problem	Time Complexity	Space	Key Recurrence / Idea
Fibonacci (DP)	O(n)	O(n)	F(n) = F(n-1) + F(n-2)
0/1 Knapsack	O(n . W)	O(n . W)	Include item i or skip it; max value
Longest Common Subsequence	O(m . n)	O(m . n)	Match = dp[i-1][j-1]+1 else max(skip)
Longest Increasing Subseq.	O(n log n)	O(n)	Binary search + patience sorting
Matrix Chain Multiplication	O(n^3)	O(n^2)	Find optimal split point k for each range
Coin Change (min coins)	O(n . amount)	O(amount)	dp[i] = min(dp[i], dp[i-coin]+1)
Edit Distance	O(m . n)	O(m . n)	Insert/delete/replace with cost 1
Rod Cutting	O(n^2)	O(n)	Cut rod at each position; max revenue

Important Formulas You Must Memorise

Exponentiation by Squaring — x^n: split as (x^(n/2))^2 if n is even, or x.(x^(n-1)) if n is odd -> O(log n)

Minimum Swaps to Sort — Answer = n minus (number of cycles in the permutation graph of the array)

T(n) = 2T(n-1) + c — Solves to O(2^n) — geometric series; each level doubles the work

T(n) = T(n-1) + n — Solves to O(n^2) — arithmetic series; sum of 1 to n

T(n) = T(sqrt(n)) + c — Solves to O(log log n) — applies repeated square root

Dijkstra with adjacency matrix — Always O(V^2) regardless of edge count — most common PGEE question

05 Discrete Mathematics & Probability

Logic · Set Theory · Counting · Graph Theory · Probability

Discrete Mathematics covers the mathematical foundations of computer science. PGEE tests propositional logic, counting techniques, probability using Bayes' theorem, and fundamental graph theory. These questions are often worth quick easy marks if you know the formulas.

Propositional Logic — Truth Values & Key Identities

Connective	Symbol	p=T, q=F gives	Only FALSE when	Key Identity
AND	p AND q	FALSE	Both needed to be True	p AND 0 = 0 | p AND 1 = p
OR	p OR q	TRUE	Both are False	p OR 0 = p | p OR 1 = 1
NOT	NOT p	FALSE	p is True	NOT(NOT p) = p
Implication	p -> q	TRUE	p=True and q=False ONLY	p->q equiv NOT p OR q
Biconditional	p <-> q	FALSE	p and q have diff values	p<->q equiv (p->q) AND (q->p)
XOR	p XOR q	TRUE	p and q are equal	p XOR q = (p OR q) AND NOT(p AND q)

Contrapositive of (p -> q) is (NOT q -> NOT p) — logically equivalent. Converse (q -> p) and Inverse (NOT p -> NOT q) are NOT equivalent to the original.

Counting Formulae

Formula	Expression	Use Case
Permutation P(n,r)	n! / (n-r)!	Ordered selection, no repetition
Combination C(n,r)	n! / (r! . (n-r)!)	Unordered selection, no repetition
With Repetition (ordered)	n^r	r items from n types, order matters
Multiset (unordered)	C(n+r-1, r)	r items from n types, order does not matter
Circular Arrangement	(n-1)!	n distinct objects around a circle
Derangements D(n)	n! . sum[(-1)^k / k!] for k=0..n	Permutations with no element in original position
Catalan Number C(n)	C(2n, n) / (n+1)	BST count, valid bracket sequences, Dyck paths
Inclusion-Exclusion	|A+B+C| = |A|+|B|+|C| - |AB| - |AC| - |BC| + |ABC|	Union of three sets
Stirling 2nd kind S(n,k)	—	Partition n labeled elements into k unlabeled groups

Probability — Core Rules & Distributions

Core Probability Rules

- P(A) = (favourable outcomes) / (total outcomes)
- P(A') = 1 - P(A) (complement rule)
- P(A or B) = P(A) + P(B) - P(A and B)
- P(A and B) = P(A) . P(B|A) (product rule)
- If A,B independent: P(A and B) = P(A) . P(B)
- Conditional: P(A|B) = P(A and B) / P(B)
- Bayes: P(A|B) = P(B|A) . P(A) / P(B)

Distributions & Statistics

- Expected value: E[X] = sum of (x . P(X=x))
- Variance: Var(X) = E[X^2] - (E[X])^2
- Binomial: P(X=k) = C(n,k) . p^k . (1-p)^(n-k)
- Mean of Binomial = n.p | Variance = n.p.(1-p)
- Geometric: P(X=k) = (1-p)^(k-1) . p
- Uniform: P(X=x) = 1/n for each of n outcomes
- Bayes' theorem is the most tested probability topic

Graph Theory — Key Theorems

Theorem / Concept	Statement	Implication
Handshaking Lemma	Sum of all vertex degrees = 2 |E|	Number of edges = half the degree sum
Euler Path	Exists if graph has exactly 0 or 2 odd-degree vertices	0 odd-degree vertices = Euler circuit
Euler Circuit	Exists iff every vertex has even degree	Graph must also be connected
Hamiltonian Path	Visits each vertex exactly once	No simple polynomial check; NP-complete
Chromatic Number x(G)	Minimum colours for valid vertex colouring	Bipartite graphs: x = 2 | K_n: x = n
Planar Graph (Euler)	V - E + F = 2 (vertices, edges, faces)	Simple planar: E <= 3V - 6
Complete Graph K_n	E = n(n-1)/2 | x(K_n) = n	Every vertex connected to every other
Tree (n vertices)	Exactly n-1 edges, connected, no cycle	Spanning tree of G: connects all V, no cycle
Bipartite Graph	No odd-length cycles; 2-colourable	Chromatic number = 2

06 Relational Databases

Normalisation · Functional Dependencies · SQL · Transactions

DBMS appears in 6 of 8 PGEE years with 4 to 6 questions. The three most tested areas are: (1) identifying the highest normal form given functional dependencies, (2) writing or analysing SQL queries with JOINs and GROUP BY, and (3) knowing the difference between DDL and DML commands. The DROP vs DELETE trap appears almost every year.

Normal Forms — How to Identify Each One

Normal Form	Violated When	How to Fix	Example Violation
1NF	Attribute is not atomic (multi-valued or composite)	Make all attributes atomic	Phone = {123, 456} in one cell
2NF	Non-prime attribute depends on part of a composite key (partial FD)	Remove partial FDs — split the table	OrderID, ProductID -> ProductName (ProductName depends only on ProductID)
3NF	Non-prime attribute depends on another non-prime attribute (transitive FD)	Remove transitive FDs — split the table	EmpID -> DeptID -> DeptName (DeptName depends on DeptID, not EmpID)
BCNF	A functional dependency X -> Y exists where X is NOT a superkey	Decompose until every determinant is a superkey	A -> C in R(A,B,C,D) where AB is the key and A alone is not a superkey
4NF	Multi-valued dependency: X ->-> Y exists where X is not a superkey	Remove multi-valued dependencies	EmpID ->-> Skill and EmpID ->-> Hobby (stored in same relation)

Functional Dependencies — Key Concepts

FD Fundamentals

- ◆ Closure X+: set of all attributes FD-determined by X
- ◆ If X+ = all attributes, then X is a superkey
- ◆ Candidate key: a minimal superkey (no proper subset is also a superkey)
- ◆ Prime attribute: belongs to at least one candidate key
- ◆ Non-prime attribute: does not belong to any candidate key
- ◆ Armstrong's Axioms: Reflexivity, Augmentation, Transitivity

Keys and Decomposition

- ◆ Finding Candidate Keys: compute closure of each subset
- ◆ Decomposition must be lossless: the FD must hold on the common attribute
- ◆ Dependency preservation: every FD in F must appear in some piece
- ◆ BCNF may not preserve all dependencies; 3NF always can
- ◆ A->C, B->D in R(A,B,C,D): AB is the key; C depends only on A = partial FD = 2NF violation
- ◆ Check BCNF: for each FD X->Y, verify X is a superkey

SQL Command Reference

Command	Category	What It Does
SELECT	DQL	Retrieves rows: SELECT col FROM tbl WHERE cond GROUP BY col HAVING cond ORDER BY col
INSERT	DML	Adds rows: INSERT INTO tbl(col1, col2) VALUES (v1, v2)
UPDATE	DML	Modifies rows: UPDATE tbl SET col = val WHERE cond
DELETE	DML	Removes rows (keeps structure): DELETE FROM tbl WHERE cond
DROP	DDL	Removes entire table including structure: DROP TABLE tbl — THIS IS DDL, NOT DML!
TRUNCATE	DDL	Removes all rows but keeps structure: TRUNCATE TABLE tbl — faster than DELETE
ALTER	DDL	Modifies schema: ALTER TABLE tbl ADD/DROP/MODIFY column
CREATE	DDL	Creates new table, index, or database object
INNER JOIN	DQL	Returns only rows where both tables have a match
LEFT JOIN	DQL	All rows from the left table; NULL for right side if no match
RIGHT JOIN	DQL	All rows from the right table; NULL for left side if no match
GROUP BY	DQL	Groups rows with the same value; use with aggregate functions
HAVING	DQL	Filters groups (like WHERE but applied after GROUP BY)
LIKE	DQL	Pattern matching: '%abc%' = contains | 'a_c' = 3 chars starting a ending c

Transactions — ACID Properties & Serializability

Property	Meaning	Enforced By
Atomicity	All operations complete or none do (no partial updates)	Undo log, rollback
Consistency	Database moves from one valid state to another	Constraints, triggers
Isolation	Concurrent transactions do not interfere with each other	Locks, 2PL, MVCC
Durability	Committed changes survive system crashes	Redo log, WAL, checkpointing
Conflict Serializable	Schedule equivalent to some serial execution order	Precedence graph must be acyclic
Two-Phase Locking	Growing phase: acquire locks only. Shrinking phase: release locks only	Guarantees conflict serializability
Clustered Index	Physically orders data on disk according to the index key	Only ONE clustered index per table
Non-Clustered Index	Separate structure with pointer to actual data row	Multiple allowed per table

07 Computer Networks

OSI & TCP/IP · IPv4 · TCP vs UDP · Routing · Protocols · Throughput

Computer Networks appears in every PGEE year. Subnetting and IP fragmentation are almost guaranteed. TCP slow start, ALOHA throughput, and protocol port numbers are frequently tested. GATE PYQs on fragmentation offset appear with changed numbers — know the formula.

OSI vs TCP/IP Model — Layer Mapping

OSI Layer	No.	TCP/IP Layer	Key Protocols	PDU Name
Application	7	Application	HTTP(80), HTTPS(443), FTP(21), SSH(22), SMTP(25), DNS(53)	Data
Presentation	6	Application	SSL/TLS, JPEG, MPEG, ASCII encoding	Data
Session	5	Application	NetBIOS, RPC, PPTP	Data
Transport	4	Transport	TCP(segments), UDP(datagrams), SCTP	Segment
Network	3	Internet	IP, ICMP, IGMP, ARP, OSPF, BGP	Packet
Data Link	2	Network Access	Ethernet, Wi-Fi (IEEE 802.11), PPP, ARP	Frame
Physical	1	Network Access	Cables, Hubs, Repeaters, Modems	Bits

IPv4 Address Classes

Class	First Octet Range	Default Subnet Mask	Networks	Hosts/Network	Private Range
A	1 – 126	255.0.0.0 (/8)	2^7 = 126	2^24 - 2	10.0.0.0/8
B	128 – 191	255.255.0.0 (/16)	2^14 = 16384	2^16 - 2	172.16.0.0/12
C	192 – 223	255.255.255.0 (/24)	2^21	2^8 - 2 = 254	192.168.0.0/16
D	224 – 239	—	—	—	Multicast
E	240 – 255	—	—	—	Reserved / Experimental

Subnetting hosts formula — Usable hosts = 2^(host_bits) - 2 (subtract network and broadcast)

Fragment Offset formula — Offset = (starting byte position) / 8 — always divide by 8

More Fragment (MF) flag — MF = 1 on all fragments EXCEPT the last one (last fragment: MF = 0)

Loopback address — 127.0.0.1 — used for local testing; never transmitted on network

TCP vs UDP — Detailed Comparison

Feature	TCP	UDP
Connection model	Connection-oriented (3-way handshake)	Connectionless (no handshake)
Reliability	Guaranteed delivery with ACKs	No guaranteed delivery
Ordering	In-order delivery guaranteed	No ordering guarantee
Error detection	Checksum + retransmission	Checksum only
Header size	20 to 60 bytes	8 bytes (fixed)
Speed	Slower due to overhead	Faster, lower latency
Flow control	Sliding window mechanism	None
Congestion control	Slow start, AIMD, fast retransmit	None — application handles it
3-way handshake	SYN -> SYN-ACK -> ACK (3 packets)	Not applicable
Use cases	HTTP, HTTPS, FTP, SSH, SMTP, POP3	DNS, DHCP, VoIP, Video streaming

Key Protocols, Ports & Throughput Formulas

Protocol / Formula	Details
HTTP / HTTPS	Port 80 / 443 | Stateless | HTTPS = HTTP + TLS/SSL encryption
FTP	Port 21 (control) + 20 (data) | File transfer protocol
SMTP	Port 25 | For sending email | Server-to-server email transfer
POP3 / IMAP	Port 110 / 143 | For retrieving email from server
DNS	Port 53 | UDP for queries | Translates domain names to IP addresses
DHCP	Port 67/68 | DORA sequence: Discover -> Offer -> Request -> Acknowledge
ARP	Maps IP address to MAC address | Broadcast request, unicast reply
ICMP	Used by ping and traceroute | No port number | Error reporting
SSH	Port 22 | Encrypted remote login (replaces Telnet port 23)
Slotted ALOHA	Maximum throughput = 1/e = 36.8% of channel capacity
Pure ALOHA	Maximum throughput = 1/(2e) = 18.4% of channel capacity
TCP Slow Start	Congestion window (CWND) doubles every RTT until threshold is reached
CWND after n RTTs	CWND = 2^n (starting from 1: after 1 RTT=2, 2 RTTs=4, 3 RTTs=8...)
Transmission time Tt	Tt = L / B where L = packet size in bits, B = bandwidth in bps
Propagation delay Tp	Tp = d / v where d = distance, v = propagation speed
Stop-and-Wait Eff.	Efficiency = 1 / (1 + 2a) where a = Tp / Tt

08 Operating Systems

Processes · Scheduling · Memory · Synchronisation · Deadlock

Operating Systems is tested in every PGEE year with 4 to 6 questions. The most frequently tested areas are page table mechanics (dirty bit, valid bit), semaphore behavior, CPU scheduling, page replacement algorithms, and deadlock theory. The semaphore wait() blocking behavior and Bakery algorithm are classic PGEE questions.

CPU Scheduling Algorithms

Algorithm	Preemptive ?	Selection Rule	Best For	Key Problem
FCFS	No	First arrived is served first	Simple batch jobs	Convoy effect: short jobs wait behind long ones
SJF	No	Shortest CPU burst time goes first	Minimum avg wait	Starvation of long processes
SRTF	Yes	Shortest remaining time is chosen	Optimal avg waiting	Starvation; high overhead
Round Robin (RR)	Yes	Fixed time quantum, cyclic rotation	Interactive users	High context switch if quantum too small
Priority	Both	Highest priority process is chosen	Real-time systems	Starvation — solved by aging (gradually increase priority)
MLFQ	Yes	Multiple queues; priority decreases over time	General purpose	Complex; approximates SJF without knowing burst time

Round Robin is the BEST scheduling algorithm for interactive processes (time-sharing systems). This is the most commonly asked scheduling question in PGEE.

Memory Management — Key Concepts

Address Translation

- ◆ Logical address: generated by the CPU during execution
- ◆ Physical address: actual location in main memory (RAM)
- ◆ MMU (Memory Management Unit): maps logical to physical address
- ◆ Paging: fixed-size pages (logical) mapped to frames (physical)
- ◆ Segmentation: variable-size named segments (code, data, stack)
- ◆ TLB (Translation Lookaside Buffer): fast cache for recent page table entries
- ◆ Page table overhead: (VA_space / page_size) * PTE_size * num_processes

Page Table Bits & Problems

- ◆ Dirty bit: set when a page is modified — must write back to disk before eviction
- ◆ Valid bit: page is currently loaded in physical memory
- ◆ Reference bit: page has been accessed recently (used by Clock algorithm)
- ◆ Internal fragmentation: wasted space WITHIN a frame (paging problem)
- ◆ External fragmentation: gaps BETWEEN allocated blocks (segmentation problem)
- ◆ Thrashing: CPU spends more time swapping pages than executing processes
- ◆ Working set: set of pages a process actively uses in a time window

Page Replacement Algorithms

Algorithm	Rule	Belady's Anomaly?	Practical?
OPT	Replace the page that will not be used for the longest time in future	No	No — requires future knowledge
LRU	Replace the page that was least recently used	No	Yes — approximated in practice
FIFO	Replace the page that was loaded earliest into memory	YES	Yes — simplest to implement
LFU	Replace the page that has been used least frequently	No	Yes — needs frequency counters
Clock (NRU)	Circular scan; replace first page with reference bit = 0	No	Yes — practical LRU approximation

FIFO is the ONLY page replacement algorithm that suffers Belady's Anomaly — adding more frames can sometimes cause MORE page faults. OPT and LRU do not suffer this.

Synchronisation — Semaphores & Mutual Exclusion

Concept	Behaviour / Rule
Semaphore wait(s)	If s > 0: decrement s and continue. If s <= 0: the calling thread BLOCKS immediately.
Semaphore signal(s)	Increment s by 1. If any thread was blocked, wake one of them up.
Binary Semaphore	Value can only be 0 or 1. Equivalent to a mutex for mutual exclusion.

Concept	Behaviour / Rule
Counting Semaphore	Value can be any non-negative integer. Used for managing a pool of resources.
Mutex	Only the thread that locked a mutex can unlock it. Ownership concept applies.
Monitor	High-level construct. Only one thread can execute inside a monitor at a time.
Peterson's Solution	Software-only mutual exclusion for 2 processes using flag[] array and turn variable.
Bakery Algorithm	Token-based algorithm. Ensures fairness (no starvation) for N processes. Does not improve throughput.
Deadlock: 4 Conditions	Must have ALL FOUR simultaneously: Mutual Exclusion + Hold and Wait + No Preemption + Circular Wait
Deadlock Prevention	Break at least one of the 4 necessary conditions to prevent deadlock entirely.
Banker's Algorithm	Resource allocation avoidance algorithm. Checks if a safe sequence exists before granting resources.
Safe State	A sequence exists in which every process can eventually finish. Safe state does NOT mean no deadlock occurred yet.
fork() return values	Parent process receives the child's PID (> 0). Child process receives 0. Error returns -1.
n fork() calls	Creates exactly 2^n - 1 child processes. Total processes including parent = 2^n.

Master Cheat Sheet — Must-Know Facts for PGEE 2026

01 Digital Logic

- JK flip-flop equation: Q* = J.Q' + K'.Q
- Mod-N counter: minimum FFs = ceiling of log base 2 of N
- Johnson counter: 2N states for N flip-flops
- De Morgan: (A.B)' = A' + B' and (A+B)' = A'.B'
- Full adder sum: A XOR B XOR Cin carry: AB + BCin + ACin
- Boolean: X(X+Y) = X | X + X.Y = X | X.X' = 0

02 C Programming

- printf() return type = int (returns characters printed)
- calloc() and malloc() return void* (not int* or char*)
- --10 cannot decrement a literal; parsed as -(-10) = 10
- static local variables are stored in the Data segment, initialised to 0
- **arr on a 2D array gives arr[0][0] (first element)
- n fork() calls create exactly 2^n - 1 child processes

03 Data Structures

- Build heap from array: O(n) | n individual inserts: O(n log n)
- AVL min nodes at height h: N(h) = N(h-1) + N(h-2) + 1
- Number of distinct BSTs with n keys = Catalan(n) = C(2n,n)/(n+1)
- Double hashing eliminates BOTH primary and secondary clustering
- Merge sort is the best sorting algorithm for linked lists
- Adjacency matrix: O(1) edge check | Adj list: O(V+E) traversal

04 Algorithms

- Dijkstra with adjacency matrix: always O(V^2) regardless of edges
- Bellman-Ford: handles negative weights in O(V.E) time
- Master Theorem case 2: T(n) = 2T(n/2) + n -> O(n log n)
- T(n) = 2T(n-1) + c solves to O(2^n) via geometric series
- Exponentiation by squaring: x^n in O(log n) time
- Min swaps to sort = n minus number of cycles in permutation graph

05 Discrete Math & Probability

- Bayes theorem: P(A|B) = P(B|A) . P(A) divided by P(B)
- Handshaking lemma: sum of all degrees = 2 times number of edges
- Euler path exists if the graph has exactly 0 or 2 odd-degree vertices
- C(n,r) = n!/(r!.(n-r)!) | Catalan(4) = 14 (BSTs with 4 keys)
- Implication p->q is FALSE only when p=True and q=False
- Tree with n vertices has exactly n-1 edges; bipartite x(G) = 2

06 DBMS

- DROP is DDL (deletes structure + data) — NOT a DML command!
- BCNF: for every FD X->Y, X must be a superkey
- 2NF: no partial functional dependency on a composite key
- 3NF: no transitive FD from non-prime to non-prime attribute
- Conflict serializable schedule: precedence graph must be acyclic
- Clustered index physically orders data on disk; only one per table

07 Computer Networks

◆ Slotted ALOHA max throughput = 1/e = 36.8% | Pure ALOHA = 1/(2e)

◆ Fragment offset = (byte start position) divided by 8

◆ TCP slow start: CWND doubles every RTT; after 4 ACKs CWND = 16

◆ Class C: 2^21 networks | 254 usable hosts per network

◆ IP is connectionless | TCP is connection-oriented

◆ Transmission time = size / bandwidth | Tp = distance / speed

08 Operating Systems

◆ Semaphore wait(s=0): calling thread BLOCKS immediately

◆ Dirty bit means the page was modified and must be written back to disk

◆ FIFO is the ONLY page replacement algorithm with Belady's anomaly

◆ Round Robin is the best scheduling algorithm for interactive processes

◆ Bakery algorithm ensures fairness; it does NOT improve throughput

◆ Safe state means a completion sequence exists — not that no deadlock happened

FINAL EXAM TIPS

- GATE PYQs (2000–2023) are the single best resource — 4 to 5 appear in PGEE verbatim, even with same options.
- No negative marking in PGEE: attempt every question. Blank answers guarantee zero marks.
- Last 2 weeks before exam: stop learning new topics. Focus only on revision and mock tests.
- Time management: do not spend more than 90 seconds on any single question — skip and return.
- Scientific calculator is allowed — similar to the GATE online calculator. Practice using it.